Fifteenth Implementation Iteration: Space Allocation

with Request Priority Based on User Role

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this fifteenth implementation iteration is to extend the system by introducing a priority mechanism for requests. In real-world scheduling scenarios, multiple requests may arrive at the same time and compete for the same resource.

To support future optimization and conflict resolution strategies, each request is now assigned a priority value derived from the role of the requester. This provides a structured basis for comparing requests and determining processing order in later iterations.

2. Changes from Iteration 14

Compared with the fourteenth iteration, the following changes were introduced:

- Added a new attribute `priority` to the abstract `Request` class
- Introduced a mapping from `UserRole` to numerical priority values
- Automatically assigned priority during request construction based on requester role
- Extended output functions to display request priority
- Preserved all existing rules and rule engine structure

3. Scenario

A user submits a request for a resource. The system assigns a priority value to the request based on the role of the requester.

For example:

- Administrator requests have the highest priority
- Instructor requests have higher priority than staff and students
- Student requests have the lowest priority

The system processes each request as before, but now each request carries a priority value that can be used for future scheduling decisions.

4. Modules and Their Tasks

A. Request Management Module

- Assign a priority to each request based on the requester role
- Store the priority as part of the `Request` object
- Provide access to the priority through a getter method

B. Resource Management Module

- Manage all supported space types without modification

C. Rule Evaluation Module

- Apply existing rules using the rule engine
- Priority is not yet used in rule evaluation

D. Allocation Management Module

- Create allocations as before
- Preserve existing allocation logic

5. Iteration Comparison

Iter.	Requests	Resources	Rules Added
1–4	OTR Only	Class → Lab → Meet	Availability + Cap.
5–6	OTR + RR	Class + Lab + Meet	(Redesign)
7	OTR + RR	Class + Lab + Meet	+ Feature
8–9	OTR + RR	Class + Lab + Meet	+ Status
10–11	OTR + RR	Class + Lab + Meet	+ Location
12	Abs. Request	Class + Lab + Meet	Polymorphism
13	Abs. Request	Class + Lab + Meet	Rule Engine
14	Abs. Request	Class + Lab + Meet	User Role Rule
15	Abs. Request	Class + Lab + Meet	**Priority Model**

Note: OTR = OneTimeRequest, RR = RecurringRequest. Rules are cumulative.

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the new `priority` attribute inside the `Request` class.

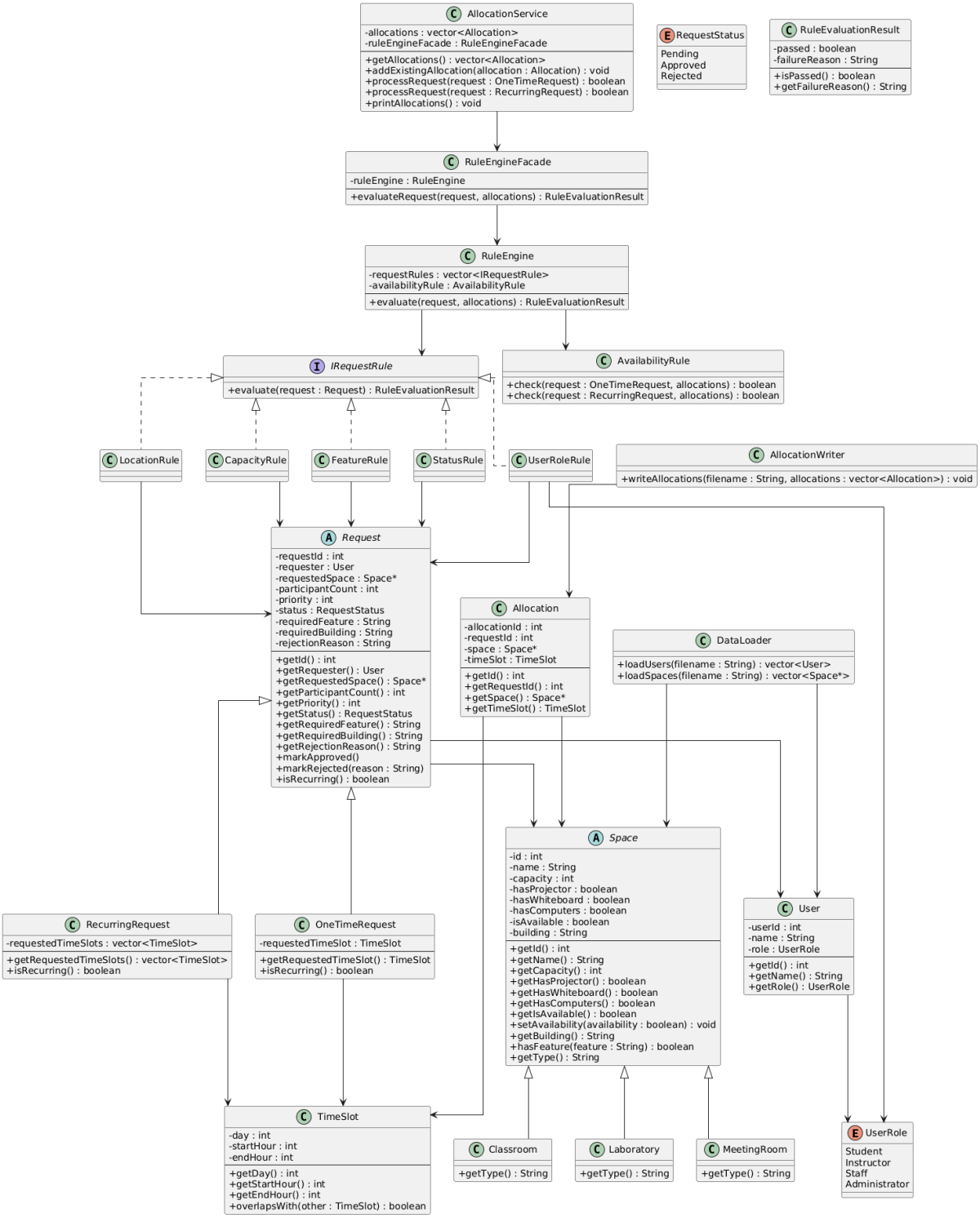

Figure 1: Updated Class Diagram with Request Priority

7. Updated Workflow Diagram

Figure 2 shows the workflow of the system. The processing logic remains unchanged, but each request now includes an associated priority value.

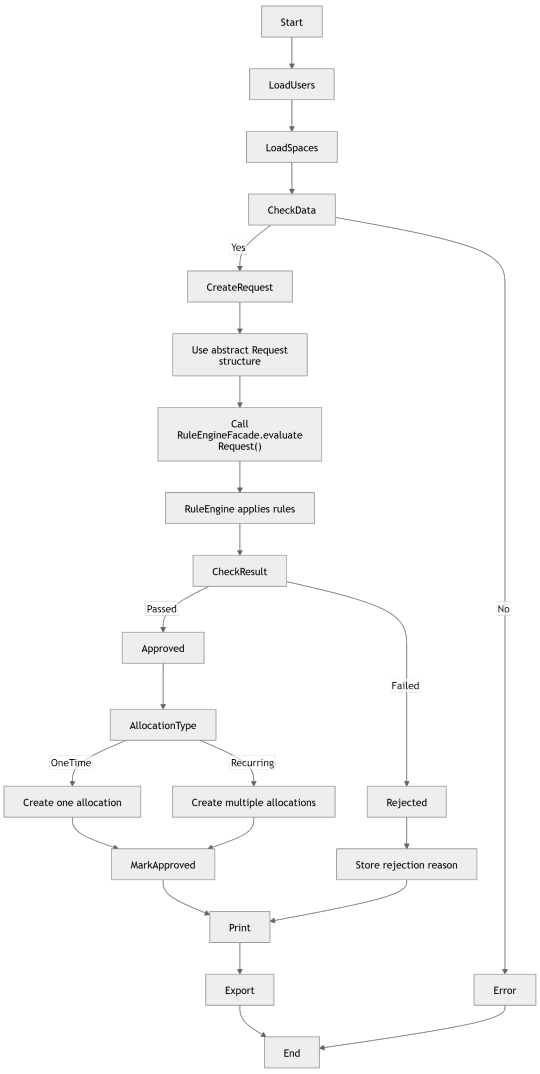

Figure 2: Workflow Including Request Priority

8. Prototype Implementation

The C++ prototype was extended by adding a `priority` attribute to the `Request` class. The priority is computed automatically using a helper function that maps the requester's role to a numerical value.

The output functions were updated to display the priority of each request. No changes were made to the rule evaluation or allocation logic.

9. Summary

This iteration introduces a priority model for requests based on user roles. Although the priority is not yet used in decision-making, it provides a foundation for future enhancements such as scheduling optimization and conflict resolution.

This iteration prepares the system for handling scenarios where multiple requests arrive simultaneously and need to be evaluated based on priority.